Instructions

Repository access. The project repository is:

https://github.com/didicayu/BDM_P1_Cayuela_Masson.git

The final P1 implementation is on branch `main`. A supervisor can clone and run it locally:

```
git clone https://github.com/didicayu/BDM_P1_Cayuela_Masson.git
cd BDM_P1_Cayuela_Masson
cp .env.example .env
```

Requirements. Docker Desktop or Docker Engine with Compose is required. Network access is needed for public data ingestion and for building the custom Airflow image, because the image downloads the Emerging Threats Open Suricata ruleset. API keys are optional except for the branches that explicitly depend on them: NVD benefits from `NVD_API_KEY`; ThreatFox uses `ABUSE_CH_API_KEY`; Shodan remains disabled unless `SHODAN_ENABLED=true`.

Build and start services.

```
docker compose build airflow-webserver airflow-scheduler airflow-init
docker compose up -d minio zookeeper kafka postgres
docker compose run --rm airflow-init
docker compose up -d airflow-webserver airflow-scheduler
```

Service UIs and endpoints: Airflow at http://localhost:8080; MinIO console at http://localhost:9001; MinIO API at http://localhost:9000; Kafka host bootstrap at `localhost:9092`. Default local credentials are defined in `.env.example`.

Run the P1 ingestion pipelines.

```
docker compose exec airflow-webserver airflow dags unpause \
  cybersecintel_dataset_artifact_ingestion
docker compose exec airflow-webserver airflow dags trigger \
  cybersecintel_dataset_artifact_ingestion
docker compose exec airflow-webserver airflow dags unpause \
  cybersecintel_api_expansion_ingestion
docker compose exec airflow-webserver airflow dags trigger \
  cybersecintel_api_expansion_ingestion
```

After at least one CTU PCAP artifact exists, enable replay in `.env` with `PCAP_REPLAY_ENABLED=true`, restart Airflow, and run:

```
docker compose up -d airflow-webserver airflow-scheduler
docker compose exec airflow-webserver airflow dags unpause \
  cybersecintel_pcap_replay
docker compose exec airflow-webserver airflow dags trigger \
  cybersecintel_pcap_replay
```

Basic verification.

```
.venv/bin/python -m unittest discover -s tests
docker compose exec kafka kafka-run-class kafka.tools.GetOffsetShell \
  --broker-list kafka:29092 --topic suricata.events --time -1
docker compose exec kafka kafka-run-class kafka.tools.GetOffsetShell \
  --broker-list kafka:29092 --topic ids.alerts --time -1
```

Raw outputs are in the MinIO bucket `landing`; Delta Lake tables are in `deltalake`. The main landing prefixes are `structured/`, `semi_structured/`, `stream/`, `unstructured/`, and `metadata/manifests/`. This instruction page is intentionally unnumbered, following the P1 statement.

CyberSecIntel
P1 Final Delivery
Contextualization, Data Ingestion and Landing Zone

Dídac Cayuela and Sindri Masson
Big Data Management – Universitat Politecnica de Catalunya

April 2026

Abstract

CyberSecIntel is a cybersecurity data platform for small Security Operations Center (SOC) workflows. The P1 deliverable implements the first stage of the required big-data architecture: contextualization, automated data ingestion, and a governed landing zone. The implementation ingests vulnerability intelligence, exploit-probability scores, IOC feeds, CTU-13 packet captures, synthetic IDS records, and replay-derived Suricata events. It combines scheduled batch ingestion with a Kafka-backed hot path, stores raw source-native assets in MinIO, and materializes queryable Delta Lake tables for metadata and structured security events. The trusted, exploitation, and final consumption layers are shown as architectural targets for P2, while P1 focuses on reproducible ingestion, raw storage, lineage, and operational automation.

1 Contextualization

1.1 Domain and operational problem

The selected domain is cybersecurity operations and vulnerability prioritization. A small SOC receives heterogeneous signals: network alerts, packet captures, malicious URL feeds, vulnerability advisories, exploit-likelihood scores, and optional exposure context from external scanners. These signals are valuable only if they can be collected consistently, traced back to origin, and later correlated across identifiers such as CVEs, IP addresses, domains, signatures, and timestamps.

In many small SOC environments, this work is fragmented. Analysts manually consult CISA KEV for exploited vulnerabilities, NVD for CVE details, EPSS for probability of exploitation, URLhaus or ThreatFox for indicators of compromise, and packet evidence or IDS logs for observed activity. The problem tackled by CyberSecIntel is therefore not only security detection, but the data-engineering foundation needed to make detection and triage repeatable: automated ingestion, immutable raw storage, metadata capture, and a path from raw evidence to stream-compatible security events.

The P1 scope is intentionally aligned with the course statement. It does not attempt to prove a final detection model or dashboard. Instead, it builds the raw and queryable foundation that later P2 trusted and exploitation zones can use. This is why packet captures remain immutable binary objects, while metadata and replay-derived events become Delta records.

1.2 Business value

The main business value is reduced time-to-triage and more consistent prioritization. A future analyst should be able to ask: which vulnerabilities are known to be exploited, which CVEs have high EPSS probability, which external indicators are active in public feeds, which PCAP artifact generated an alert, and where the original raw evidence is stored. The platform supports that value by removing manual collection steps and replacing them with repeatable Airflow tasks.

For P1, the measurable value is operational reproducibility:

- every raw payload or PCAP is stored in a stable partitioned object-store path;
- every ingested artifact has metadata describing origin, size, checksum, timestamp, and selection context;
- every source has manifest entries for auditability and reruns;
- queryable Delta tables expose catalogs and structured records without discarding the raw files;
- replay-derived events are also published to Kafka topics, proving that landed historical artifacts can feed the hot path.

1.3 Stakeholders and success criteria

The direct users are SOC analysts, security engineers, patch-management teams, and management stakeholders. Analysts need searchable evidence and alert context; engineers need reliable pipelines and clear failure modes; patch teams need vulnerability signals such as KEV and EPSS; management needs later aggregates and KPIs. The P1 success criteria are therefore: include all required data categories, automate ingestion from non-local sources, implement a real landing zone, orchestrate the workflows, containerize deployed services, and leave a clear extension path for P2.

2 Data Sources and Selection

2.1 Source mix

The source mix intentionally includes structured, semi-structured, unstructured, batch, and stream-like data. This satisfies the project requirement while also matching the cybersecurity domain: vulnerability intelligence is naturally tabular or semi-structured, IOC feeds are irregular and source-specific, and network evidence is often raw binary PCAP.

Source	Type	Access pattern	Role in the platform
CISA Known Exploited Vulnerabilities (KEV)	Structured	Scheduled JSON feed pull	Canonical exploited CVE list, used as a strong prioritization baseline.
FIRST EPSS	Structured	Paginated API pull	Daily exploit probability scores for ranking CVEs and choosing enrichment targets.
NVD CVE v2	Semi-structured	REST API with time windows and pagination	CVE descriptions, CVSS metrics, configurations, and references for enrichment.
URLhaus recent feed	Semi-structured	Public CSV feed pull	Malicious URL and host indicators for IOC context.
CIRCL Vulnerability-Lookup	Semi-structured	REST API enrichment	Additional vulnerability context for high-priority EPSS CVEs.
ThreatFox	Semi-structured	Optional authenticated API pull	IOC feed with malware infrastructure indicators; skipped when no key is available.
Shodan host lookup	Semi-structured	Optional seeded API lookup	Exposure details for selected IP indicators; disabled by default to control cost and scope.
CTU-13 PCAP artifacts	Unstructured	Remote dataset discovery and download	Raw packet evidence; kept immutable and replayed through Suricata.
Synthetic and replay-derived IDS events	Structured event stream	Python producer and Kafka publish	Hot-path scaffold and real alert/event stream for later aggregation.

2.2 Why these sources fit

CISA KEV, NVD, and EPSS represent vulnerability prioritization from complementary angles: exploited in the wild, technical detail and severity, and probability of exploitation. URLhaus, ThreatFox, CIRCL, and Shodan add IOC and enrichment context that is useful for later correlation. CTU-13 adds the unstructured network-evidence branch with realistic packet traffic and enough size to justify object storage. The synthetic stream keeps the hot path testable even when no real IDS sensor is connected, while Suricata replay proves that the unstructured branch can be converted into event telemetry.

This combination is also feasible for a course project because the sources are public, programmatically accessible, and reproducible. It avoids a purely local-file workflow: API/feed tasks access external services, CTU discovery reads a public dataset host, and replay is performed from the landed raw artifact rather than from manually copied local files.

2.3 Official source references

The implementation uses the following official pages and endpoints:

- CISA KEV catalog and JSON feed: CISA KEV catalog, CISA KEV JSON feed.
- NVD CVE API v2: NVD vulnerability API guide, NVD CVE API endpoint.
- FIRST EPSS API: FIRST EPSS API guide and FIRST EPSS endpoint.
- URLhaus recent feed and ThreatFox API: URLhaus recent CSV, ThreatFox API.
- CIRCL Vulnerability-Lookup and Shodan APIs: CIRCL documentation, Shodan API documentation.
- CTU-13 dataset reference and artifact host: CTU-13 reference, CTU public dataset host.

3 Architecture Design

3.1 High-level design

The architecture follows the course framework: data ingestion, raw data storage in a landing zone, future trusted zone, future exploitation zone, and data consumption. P1 implements ingestion and the landing zone, while also preparing the technical interfaces that P2 will consume. The trusted and exploitation zones remain black boxes in the diagram because they are not part of the P1 implementation, but the data contracts they will use are already visible: Delta tables, Kafka topics, raw object paths, and manifest lineage.

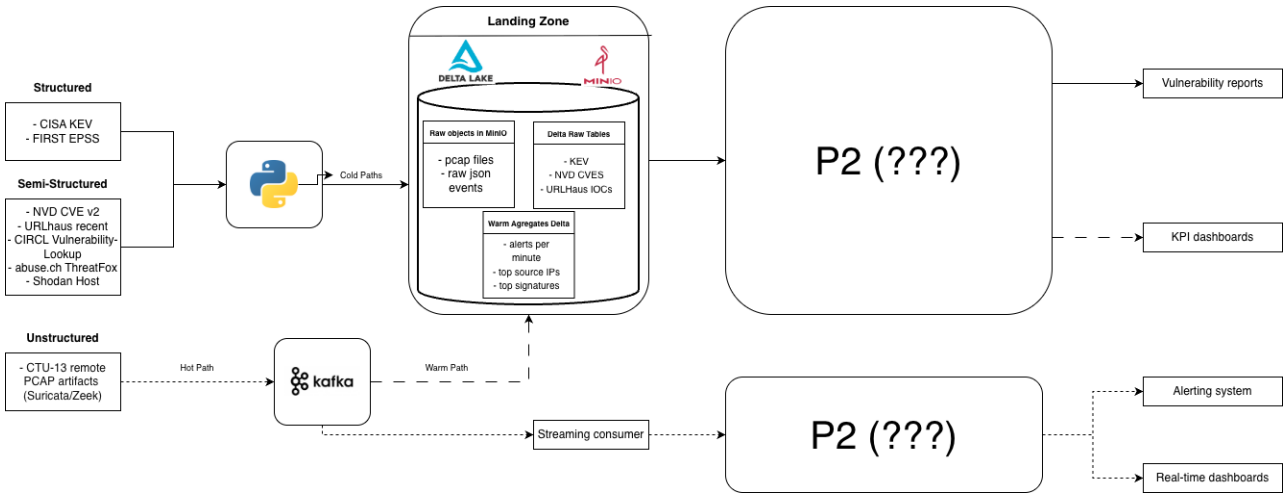

Figure 1: CyberSecIntel architecture from the P1 design. The final implementation refines the unstructured branch with automated CTU-13 ingestion, Suricata replay, Delta catalogs, and Kafka publication.

The diagram keeps the same conceptual separation as the first design: external cybersecurity sources feed ingestion components; ingestion lands raw data in the lakehouse; stream and replay paths provide operational events; future trusted and exploitation stages will clean, integrate, aggregate, and serve the data. The final implementation changed one important practical detail from the first draft: the manual CIC artifact idea was replaced by automated CTU-13 discovery and download, because the P1 statement explicitly rewards ingestion from non-local environments.

3.2 Cold, hot, and warm paths

Cold path. Public vulnerability and IOC feeds are ingested in batches. Their raw payloads are stored in MinIO using source-native formats, and flattened records are merged into Delta tables when appropriate. CTU-13 PCAP files also belong to the cold path because they are large immutable evidence artifacts that should remain queryable only through metadata, not through direct binary table conversion.

Hot path. Kafka carries time-sensitive security events. P1 implements two producers: a bounded synthetic IDS producer for repeatable testing and the Suricata replay branch for real replay-derived events. The replay branch publishes all normalized EVE events to `suricata.events` and alert-compatible records to `ids.alerts`. This gives P2 a real message interface for near-real-time consumers.

Warm path. The landing setup reserves `warm/stream_aggregates/`. P1 does not yet implement rolling aggregations, but the stored events and Kafka topics make the planned work straightforward: P2 can compute alerts per minute, top source IPs, signatures by severity, or vulnerability-linked risk summaries.

3.3 P1 and P2 boundary

The main P1 responsibility is to make raw data available, traceable, and reproducible. The system therefore avoids prematurely imposing final analytical schemas on every source. The P2 trusted zone will apply stronger schema validation, entity resolution across CVEs and IOCs, quality checks, and deduplication. The exploitation zone will then produce dashboards, prioritization tables, and SOC-facing aggregates. This boundary keeps P1 focused on data management rather than final analytics.

4 Data Ingestion

4.1 Airflow orchestration

Three Airflow DAGs automate the P1 flows:

- `cybersecintel_dataset_artifact_ingestion`: prepares landing prefixes, discovers CTU-13 scenario folders from the public host, downloads selected PCAP artifacts, writes sidecars and manifests, and materializes the `pcap_artifacts` catalog.
- `cybersecintel_api_expansion_ingestion`: ingests KEV, NVD, URLhaus, EPSS, CIRCL, optional ThreatFox, optional Shodan, and the synthetic IDS stream scaffold.
- `cybersecintel_pcap_replay`: intentionally triggered after PCAP artifacts exist; builds the catalog, selects replay candidates, stages artifacts, runs Suricata, persists EVE output, writes Delta rows, and publishes Kafka messages.

Airflow provides task dependencies, retry behavior, execution logs, and operational visibility. Replay is separated into its own DAG rather than being hidden inside the CTU ingestion DAG because offline PCAP processing is heavier and should be triggered deliberately. The guard `PCAP_REPLAY_ENABLED=true` prevents accidental full-corpus replay during demos.

4.2 Batch API and feed ingestion

Each public source has a dedicated Python adapter under `ingestion/batch/`. The common pattern is:

1. request the public API or feed using bounded pagination, time windows, retries, and timeouts;
2. write the raw source payload to MinIO under a partitioned path;
3. write a metadata sidecar and append a manifest entry;
4. flatten the records that are useful for query and merge them into a Delta table.

This design deliberately separates raw preservation from query-oriented representation. NVD, ThreatFox, and Shodan can change shape over time, so the raw JSON is the authoritative record. Delta tables are a convenience layer for P1/P2 processing, not a replacement for the original source data. KEV and EPSS are more stable and are therefore easier to merge by natural keys such as CVE and date.

4.3 CTU-13 artifact ingestion

The unstructured branch is implemented through remote CTU discovery instead of manual file staging. The downloader inspects the public dataset directory, matches CTU botnet scenario folders numbered `42` through `54`, inspects available artifacts, and selects a preferred PCAP according to `PCAP_SOURCE_PROFILE`. The default `subset` profile is demo-safe and prefers the botnet-only plain

.pcap when available; the full profile processes one artifact per scenario and can select compressed .pcap.bz2 files.

Every landed PCAP receives a metadata sidecar with source URL, scenario number, artifact name, content type, size, checksum, profile, selection rule, and retrieval timestamp. The raw PCAP remains immutable in the bronze layer. The pcap_artifacts Delta table makes the binary corpus queryable without converting PCAP bytes into tabular data.

4.4 PCAP replay and Suricata normalization

The replay branch turns selected landed packet captures into replay-derived security telemetry. The chosen replay engine is Suricata offline mode, executed with suricata -r <pcap>. The Airflow image includes Suricata, suricata-update, and the Emerging Threats Open ruleset, and the Docker build validates the generated suricata.rules file with suricata -T.

The replay task stages a selected artifact to /tmp/cybersecintel/<dag>/<run>/, verifies its checksum, optionally decompresses .pcap.bz2 when explicitly enabled, runs Suricata, and writes eve.json back to MinIO as eve.jsonl. The parser keeps the configured EVE event types: alert, flow, dns, http, and tls. Normalized records contain stable fields such as event type, timestamp, source/destination IP and port, protocol, application protocol, signature, severity, category, flow id, packet count, scenario, artifact checksum, replay run id, and ingestion date.

Alerts are additionally projected into the existing IDS event shape so that replay output and stream events share the same downstream interface. This design is more stable than live packet replay for P1 because it avoids timing, interface, and privilege issues while still proving that raw PCAP artifacts can produce operational telemetry.

4.5 Streaming ingestion

Kafka is deployed with Zookeeper and exposes localhost:9092 to the host and kafka:29092 to containers. P1 uses Kafka for the hot path in two ways. The synthetic IDS task writes bounded alert records for repeatable tests. The replay module publishes normalized Suricata records to suricata.events and alert-compatible records to ids.alerts. With PCAP_REPLAY_KAFKA_STRICT=true, publish failures fail the replay task, which is useful because it proves the hot-path handoff rather than silently ignoring it.

5 Landing Zone

5.1 Storage choice

The landing zone is a local lakehouse. MinIO is used as the S3-compatible object store for raw bronze objects. Delta Lake is used as the open table format for queryable records, metadata catalogs, replay-run records, and normalized events. This choice satisfies the project recommendation to use object storage and an open table format while keeping the stack deployable on a laptop with Docker Compose.

The design follows a read-first, model-later philosophy. Raw data is preserved before transformation, which is important for reproducibility and for future schema changes. The Delta layer adds ACID commits, merge semantics, Parquet storage, schema tracking, and versioned transaction logs. In a cloud deployment, the same storage abstraction could point to an S3-compatible service rather than local MinIO.

5.2 Object layout

The `landing` bucket is organized by data family, source, and logical partitions. The main prefixes are:

Prefix	Purpose
`structured/kev/`	Raw CISA KEV feed, partitioned by ingestion date.
`structured/epss/`	Raw FIRST EPSS API pages, partitioned by ingestion date and page number.
`semi_structured/nvd/`	Raw NVD CVE API pages from bounded time windows.
`semi_structured/urlhaus/`	Raw URLhaus recent CSV feed.
`semi_structured/circl_vulnlookup/`	CIRCL enrichment responses for selected CVEs.
`semi_structured/threatfox/` and `semi_structured/shodan_seeded/`	Optional IOC and exposure enrichment branches.
`unstructured/pcap/source=ctu13/`	Immutable CTU PCAP or PCAP.BZ2 artifacts, partitioned by scenario and date.
`unstructured/pcap_replay/source=ctu13/`	Suricata EVE JSONL, partitioned by scenario, artifact hash, and date.
`stream/ids_alerts/source=suricata/`	Replay-derived alert JSONL, partitioned by date and hour.
`metadata/manifests/`	Per-source JSONL manifests for lineage, audit, and reruns.

5.3 Delta table layout

The `deltalake` bucket stores queryable Delta tables:

- `kev`, `epss`, `nvd`, `urlhaus`, `circl_vulnlookup`, `threatfox`, and `shodan_seeded` store flattened feed/enrichment records.
- `ids_alerts` is append-only and stores both synthetic IDS events and Suricata alert-shaped records.
- `pcap_artifacts` catalogs landed PCAP binaries by source, scenario, artifact name, ingestion date, and SHA-256.
- `pcap_replay_runs` stores one row per replay attempt, including status, event counts, output paths, timings, and error summaries.
- `suricata_events` stores normalized EVE records with lineage to the artifact and replay run.

Merge keys are used for stable batch sources and catalogs, while append-only writes are used for stream-style event tables. This prevents duplicate catalog rows on rerun, while preserving event history where multiple replays are meaningful.

5.4 Metadata management and idempotency

The landing zone is designed to avoid becoming a data swamp. Each raw object has a sidecar with origin and integrity metadata. Each source also appends JSONL manifest entries under `metadata/manifests/ingest_date=...`. PCAP sidecars include checksums, and replay staging verifies those checksums before Suricata runs. Replay runs receive unique `replay_run_id` values, so generated EVE events and IDS alerts can be traced back to both the original PCAP and the replay attempt that produced them.

The implementation also avoids deleting or overwriting raw PCAP artifacts after replay. Generated replay outputs are stored separately under `unstructured/pcap_replay/`. This separation is important because PCAP files are evidence, while EVE JSON and alert records are derived products.

6 Containerization and Organization

6.1 Runtime services

The deployment is containerized with Docker Compose. The stack includes MinIO for object storage, Kafka and Zookeeper for the hot path, Postgres for Airflow metadata, and Airflow webserver/scheduler/init services for orchestration. A custom Airflow image installs Python dependencies, Suricata, `suricata-update`, and the Emerging Threats Open ruleset. The build step fails fast if the rules file is missing or if Suricata cannot load the configuration.

This local stack is not a production distributed cluster, but it follows the same interfaces: S3-compatible object operations, Kafka topics, Airflow DAGs, and Delta transaction logs. That makes the design portable to managed equivalents such as S3-compatible object storage, managed Kafka, or hosted Airflow.

6.2 Code organization

The repository is organized by stage and responsibility:

- `ingestion/batch/`: source-specific public feed and API clients;
- `ingestion/datasets/`: CTU remote PCAP discovery and download;
- `ingestion/replay/`: Suricata offline replay, normalization, Delta writes, and Kafka publishing;
- `ingestion/stream/`: synthetic IDS event scaffold;
- `ingestion/common/`: shared HTTP, landing storage, metadata, and Delta helpers;
- `landing/`: landing-zone prefix setup;
- `orchestration/airflow/dags/`: Airflow DAG definitions;
- `docs/`: delivered reports, runbooks, and architecture documentation.

This separation improves reusability: adding a new source should generally mean adding a focused adapter and DAG task while reusing storage, manifest, and Delta helpers.

7 Validation and Current Results

The implementation was validated through unit tests, Docker Compose checks, Airflow runs, object-store inspection, Delta table reads, and Kafka topic inspection. Unit tests cover PCAP catalog construction, candidate selection, EVE normalization, compressed artifact staging, Kafka publishing, and replay-related edge cases. The latest test suite result is `14 tests OK`.

The most recent Suricata replay validation used CTU scenario 42 with the Emerging Threats Open ruleset. The replay DAG completed successfully and returned:

- `candidate_artifacts=1`;
- `successful_replays=1`;
- `normalized_events=106077`;
- `alert_events=2762`;
- `kafka_suricata_events=106077`;
- `kafka_ids_alerts=2762`.

The landing bucket contains `unstructured/pcap_replay/source=ctu13/.../eve.jsonl` and `stream/ids_alerts/source=suricata/ingest_date=2026-04-11/hour=14/ids_suricata.jsonl`.

The Delta tables include `pcap_artifacts`, `pcap_replay_runs`, `suricata_events`, and `ids_alerts`. Kafka offsets verified the hot-path publication: `suricata.events` contained `209403` messages after repeated test replays, and `ids.alerts` contained `2762` messages from the ruleset-backed replay. A consumed alert sample had source `suricata`, scenario `42`, severity `3`, and signature `SURICATA TCPv4 invalid checksum`.

8 Constraint Coverage

P1 requirement	Implemented response
Architecture diagram	Figure 1 shows sources, ingestion, storage, stream path, and downstream black boxes. The text explains the final implementation refinements.
Contextualization	SOC triage and vulnerability prioritization are defined, including stakeholders, problem, value, and success criteria.
Data selection	Structured, semi-structured, unstructured, and stream-like data are all included from public or reproducible sources.
Ingestion design	Batch, streaming, cold, hot, and warm paths are explicitly separated and justified.
Ingestion implementation	Airflow DAGs automate public APIs, feeds, CTU remote downloads, synthetic IDS events, and Suricata replay.
Landing design	MinIO implements the bronze data lake; Delta Lake provides lakehouse metadata and queryable tables.
Landing implementation	Raw objects, sidecars, manifests, Delta tables, replay output, and IDS alert objects are written and verified.
Orchestration	Airflow manages task order, logging, retries, guards, and manual or scheduled execution.
Containerization	Docker Compose deploys MinIO, Kafka, Zookeeper, Postgres, and custom Airflow.
Organization	Code is separated by batch, dataset, replay, stream, common helpers, landing setup, and orchestration.

9 Limitations and Next Steps

The full CTU corpus is intentionally not replayed by default because it can be large in storage, CPU, and runtime cost. The demo profile therefore uses `PCAP_REPLAY_MAX_ARTIFACTS=1`, skips compressed artifacts unless explicitly enabled, and keeps replay manually gated. ThreatFox and Shodan depend on API-key availability; Shodan is disabled by default to avoid unnecessary external calls. Kafka is currently used for publication and validation, while durable stream processing and warm aggregate consumers are reserved for P2.

The next stage will instantiate the trusted zone with stronger schemas, source-quality checks, CVE and IOC entity resolution, and replay-event validation. The exploitation zone can then produce SOC-facing tables: alert trend aggregates, top source/destination indicators, vulnerability prioritization views that combine KEV and EPSS, and traceability links from each derived insight back to raw evidence.

10 References

- Apache Airflow documentation: https://airflow.apache.org/docs/
- Apache Kafka documentation: https://kafka.apache.org/documentation/
- Delta Lake project: https://delta.io/
- MinIO object storage: https://min.io/

- Suricata command-line and EVE JSON documentation: https://docs.suricata.io/en/latest/command-line-options.html, https://docs.suricata.io/en/latest/output/eve/eve-json-output.html
- Emerging Threats Open ruleset through Suricata Update: https://suricata-update.readthedocs.io/
- CISA KEV, NVD, FIRST EPSS, URLhaus, CIRCL, ThreatFox, Shodan, and CTU-13 source references are listed in Section 2.3.